Lesson 11：Git 工作区、暂存区与本地提交

LLM Learn 大模型开发课程

2026 年 11 月

目录

1 教学目标与先修 2

2 Git 的四个位置 2

3 安全的常用命令 2

4 代码原理与函数解释 2

5 实验步骤 2

6 核心关系或公式 2

7 本课完整源码：逐行中文注释 2

8 练习与验收 4

9 执行记录与参考 4

1 教学目标与先修

会安全查看 Git 历史、工作区和分支，不在主分支上使用破坏性回退命令。

先修：Python 终端与文件操作；已安装 Git。预计 90–120 分钟。

2 Git 的四个位置

工作区保存正在修改的文件；暂存区决定下一次提交；本地仓库保存 commit 快照与父提交；远程仓库存储可共享的历史。git add 不会上传到 GitHub，git commit 不会自动远程同步。

3 安全的常用命令

git status 查看变更；git diff 查看未暂存修改；git add 指定目录；git diff –cached 核对暂存变更；git commit 生成本地快照；git log –oneline 查看历史。不要在已有仓库中重新 init，也不要在 main 上练习 reset –hard。

4 代码原理与函数解释

run_git 把 Git 子命令封装为参数数组,通过 subprocess.run(check=True,capture_output=True,text=True) 获得输出；summarize_repository 查询分支、状态、提交；main 负责解析路径和处理失败。请逐行阅读下方源文件中的中文注释。

5 实验步骤

在仓库根目录运行 python lessons/lesson11/code/inspect_git.py .；在个人练习分支修改文件，使用 git status、git diff、git add、git commit 和 git log 观察状态变化。

6 核心关系或公式

$$W \xrightarrow{\text{git add}} S \xrightarrow{\text{git commit}} C$$

7 本课完整源码：逐行中文注释

下列程序与本课 code 目录的实际源文件完全对应。每个模块、函数及关键可执行语句都配有中文说明。

```
"""Lesson 11: 检查本地 Git 仓库状态与提交；仅调用本机 git，不修改仓库。"""

# 导入 argparse，用于解析命令行中可选的仓库目录参数。
```

```
import argparse
# 导入 subprocess，以参数列表安全执行本地 Git 命令。
import subprocess
# 导入 Path，以独立于操作系统的方式管理文件路径。
from pathlib import Path


def run_git(directory: Path, *arguments: str) -> str:
    """ 在指定仓库执行 Git 子命令，返回标准输出；失败时抛出异常。"""
    # 构造完整 Git 命令；不用 shell=True，以避免命令注入。
    command = ["git", "-C", str(directory), *arguments]
    # 执行命令，检查非零返回码，并把字节解码为文本。
    completed = subprocess.run(command, check=True, capture_output=True, text=True)
    # 去掉输出头尾的换行，但不改变内部内容。
    return completed.stdout.strip()


def summarize_repository(directory: Path) -> dict[str, str]:
    """ 读取分支、工作区状态与最新提交，返回可序列化的摘要字典。"""
    # 查询 HEAD 对应的当前分支；如果是 detached HEAD 可能返回 HEAD。
    branch = run_git(directory, "rev-parse", "--abbrev-ref", "HEAD")
    # 使用 porcelain 格式读取工作区，便于程序进行后续处理。
    status = run_git(directory, "status", "--short")
    # 读取最新一次提交的短哈希和主题，不修改仓库历史。
    latest = run_git(directory, "log", "-1", "--pretty=%h %s")
    # 把三个字段合成为字典，供打印或下一课做 JSON 化。
    return {"branch": branch, "status": status, "latest_commit": latest}


def main() -> None:
    """ 解析路径、调用 Git 状态检查函数，并输出用户可读的结果。"""
    # 创建命令行解析器，并描述这个工具的用途。
    parser = argparse.ArgumentParser(description="Lesson 11: Git 仓库状态查询")
    # 接受可选目录参数；默认检查当前工作目录。
    parser.add_argument("directory", nargs="?", default=".")
    # 把命令行字符串转换为结构化参数。
    arguments = parser.parse_args()
    # 将指定目录转换为绝对路径，避免工作目录歧义。
    directory = Path(arguments.directory).resolve()
    try:
        # 读取 Git 摘要；git 不存在或目录不合法时可能失败。
        summary = summarize_repository(directory)
    except (FileNotFoundError, subprocess.CalledProcessError) as error:
        # 明确说明操作失败，而不是捏造分支和提交结果。
        parser.exit(1, f"Git 检查失败：{error}\n")
    # 逐个输出摘要字段，让读者理解 Git 的三个常用视角。
    for key, value in summary.items():
        # 对空工作区状态使用明确说明，避免误认为漏掉输出。
        print(f"{key}: {value or '(clean)'}")


if __name__ == "__main__":
    # 只在直接运行本文件时调用入口，import 时不自动检查仓库。
    main()
```

8 练习与验收

- 解释 Working Tree、Index、Commit、Remote 的区别。
- 完成一个不影响 main 的分支提交。
- 离线 test_inspect_git.py 在临时仓库中通过。

9 执行记录与参考

使用 python -m unittest discover -s tests -v 执行测试。请以本课 results/verification.md 中的真实结果和 GitHub Actions 日志为准，Mock 并非在线 API 验证。

- https://git-scm.com/docs
- https://docs.github.com/en/actions
- https://developer.mozilla.org/en-US/docs/Web/HTTP
- https://platform.openai.com/docs/api-reference/responses